SelVeri: Self-Verifying Programming Language
High Level Language Syntax and Semantics Document
v0.9.1

Yusuf Demir
Yağız Kaan Aydoğdu

May 10, 2026

1 Syntax

1.1 Lexical Categories

Identifiers must start with a letter or underscore, and continue with letters, digits, or underscores. Integer and floating-point literals are given by the following lexical categories:

$$\begin{array}{rcl} \langle \mathit{Ident} \rangle & ::= & [\texttt{A-Za-z}][\texttt{A-Za-z0-9\_}]^* \\ \langle \mathit{IntLit} \rangle & ::= & [\texttt{0-9}]^+ \\ \langle \mathit{FloatLit} \rangle & ::= & [\texttt{0-9}]^+\texttt{.}[\texttt{0-9}]^+ \end{array}$$

1.2 Types

All variables and function return values are statically typed.

$$\begin{array}{rcl} \langle \mathit{BasicType} \rangle & ::= & \texttt{Int} \mid \texttt{Float} \\ \langle \mathit{Type} \rangle & ::= & [\langle \mathit{BasicType} \rangle \mid \texttt{List}\,[\langle \mathit{BasicType} \rangle, \langle \mathit{IntLit} \rangle\,(, \langle \mathit{AExp} \rangle)^+] \mid \texttt{type}(\langle \mathit{ident} \rangle) \end{array}$$

1.3 Immediates

$$\begin{array}{rcl} \langle \mathit{Imm} \rangle & ::= & \langle \mathit{IntLit} \rangle \mid \langle \mathit{FloatLit} \rangle \mid \langle \mathit{ListLit} \rangle \\ \langle \mathit{ListLit} \rangle & ::= & [\,] \mid [\,\langle \mathit{Imm} \rangle\,(\texttt{,}\,\langle \mathit{Imm} \rangle)^*\,] \end{array}$$

1.4 Expressions

Arithmetic Expressions We declare the syntax of arithmetic expressions so that the operator precedence makes sense in the usual manner.

$$\begin{array}{rcl} \langle \mathit{AExp} \rangle & ::= & \langle \mathit{ASum} \rangle \\ \langle \mathit{ASum} \rangle & ::= & \langle \mathit{ASum} \rangle \texttt{ + } \langle \mathit{ATerm} \rangle \mid \langle \mathit{ASum} \rangle \texttt{ - } \langle \mathit{ATerm} \rangle \mid \langle \mathit{ATerm} \rangle \\ \langle \mathit{ATerm} \rangle & ::= & \langle \mathit{ATerm} \rangle \texttt{ * } \langle \mathit{AFactor} \rangle \mid \langle \mathit{ATerm} \rangle \texttt{ / } \langle \mathit{AFactor} \rangle \mid \langle \mathit{AFactor} \rangle \\ \langle \mathit{AFactor} \rangle & ::= & \texttt{-}\,\langle \mathit{AFactor} \rangle \mid \langle \mathit{AAtom} \rangle \\ \langle \mathit{AAtom} \rangle & ::= & \langle \mathit{Imm} \rangle \mid \langle \mathit{Ident} \rangle \mid \texttt{len}(\langle \mathit{Ident} \rangle) \mid \texttt{read}() \mid \langle \mathit{IndexedVal} \rangle \mid (\langle \mathit{AExp} \rangle) \\ \langle \mathit{IndexedVal} \rangle & ::= & \langle \mathit{IndexedVal} \rangle[\langle \mathit{Aexp} \rangle] \mid \langle \mathit{Ident} \rangle[\langle \mathit{Aexp} \rangle] \end{array}$$

`len` keyword defined above semantically gives us the length of a list. In the current implementation, length of a list is fixed, so it can be resolved via a lookup from the scope table. It must not be confused with a function call, as no scope switches happen while resolving this value.

Boolean Expressions Boolean expressions used in guards and specifications include arithmetic expressions as truthy values.

$$
\begin{aligned}
\langle BExp \rangle &::= \langle BOr \rangle \\
\langle BOr \rangle &::= \langle BOr \rangle \texttt{ or } \langle BXor \rangle \mid \langle BXor \rangle \\
\langle BXor \rangle &::= \langle BXor \rangle \texttt{ xor } \langle BAnd \rangle \mid \langle BAnd \rangle \\
\langle BAnd \rangle &::= \langle BAnd \rangle \texttt{ and } \langle BNot \rangle \mid \langle BNot \rangle \\
\langle BNot \rangle &::= \texttt{not } \langle BNot \rangle \mid \langle BAtom \rangle \\
\langle BAtom \rangle &::= \texttt{true} \mid \texttt{false} \mid \langle Comparison \rangle \mid \langle AExp \rangle^{1} \mid (\langle BExp \rangle) \\
\langle Comparison \rangle &::= \langle AExp \rangle \langle CompOp \rangle \langle AExp \rangle \\
\langle CompOp \rangle &::= \texttt{=} \mid \texttt{<=} \mid \texttt{<} \mid \texttt{>=} \mid \texttt{>}
\end{aligned}
$$

1.5 Specifications (Runtime Assertions)

Here the syntax is also constructed in a way that establishes operator precedence rules. In the implemented parser, $\langle Spec \rangle$ nodes are not parsed further and parser accepts any character sequence in the place of this node. This is because specifications are compiled to the intermediate language literally and parsed for the verification module starting from the IR code.[2]

$$
\begin{aligned}
\langle Spec \rangle &::= \langle SpecImp \rangle \\
\langle SpecImp \rangle &::= \langle SpecOr \rangle \texttt{ => } \langle SpecImp \rangle \mid \langle SpecOr \rangle \\
\langle SpecOr \rangle &::= \langle SpecOr \rangle \texttt{ || } \langle SpecAnd \rangle \mid \langle SpecAnd \rangle \\
\langle SpecAnd \rangle &::= \langle SpecAnd \rangle \texttt{ \&\& } \langle SpecTemp \rangle \mid \langle SpecTemp \rangle \\
\langle SpecTemp \rangle &::= \langle SpecTemp \rangle \texttt{ Since } \langle SpecUnary \rangle \mid \langle SpecTemp \rangle \texttt{ Until } \langle SpecUnary \rangle \mid \langle SpecUnary \rangle \\
\langle SpecUnary \rangle &::= \texttt{!} \langle SpecUnary \rangle \mid \texttt{Previously } \langle SpecUnary \rangle \mid \texttt{Once } \langle SpecUnary \rangle \mid \texttt{Historically } \langle SpecU \\
&\quad \mid \texttt{Next } \langle SpecUnary \rangle \mid \texttt{Eventually } \langle SpecUnary \rangle \mid \texttt{Always } \langle SpecUnary \rangle \\
&\quad \mid \texttt{Forall } \langle Ident \rangle \texttt{ : } \langle Domain \rangle \texttt{ . } \langle Spec \rangle \mid \texttt{Exists } \langle Ident \rangle \texttt{ : } \langle Domain \rangle \texttt{ . } \langle Spec \rangle \\
&\quad \mid \langle BExp \rangle \mid (\langle Spec \rangle) \\
\langle Domain \rangle &::= [\langle IntLit \rangle \texttt{ ... } \langle IntLit \rangle) \mid [\langle FloatLit \rangle \texttt{ , } \langle FloatLit \rangle] \mid (\langle FloatLit \rangle \texttt{ , } \langle FloatLit \rangle) \\
&\quad [\langle FloatLit \rangle \texttt{ , } \langle FloatLit \rangle) \mid (\langle FloatLit \rangle \texttt{ , } \langle FloatLit \rangle] \\
&\quad \mid \langle Ident \rangle \mid \langle BasicType \rangle \mid \texttt{Var[} \langle BasicType \rangle \texttt{]} \mid \langle ListLit \rangle
\end{aligned}
$$

[1] We interpret results of arithmetic expressions as boolean values where needed using equality to zero as in other imperative languages like C. This is useful for being able to write statements like while x, where x is an integer variable.

[2] This approach might lead to the generation of broken IR code as a non-valid specification is accepted by the parser. However, in the final design, IR generation is planned to be in memory (i.e. no IR code file to be generated). Thus, in the final implementation, after the generation of IR code of the input program, all specifications will be parsed in the memory with position information before the beginning of execution. This way, a compiler error can still be thrown for invalid specification statements.

1.6 Statements, Functions, and Programs

Top-Level Statements

$$
\begin{array}{rcl}
\langle StmtSeq \rangle & ::= & \langle Stmt \rangle^* \\
\langle Stmt \rangle & ::= & \langle TermStmt \rangle \mid \langle IfStmt \rangle \mid \langle WhileStmt \rangle \mid \langle AssertStmt \rangle \mid \texttt{;} \\
\langle TermStmt \rangle & ::= & \langle Decl \rangle \texttt{;} \mid \langle Assign \rangle \texttt{;} \mid \langle ListAssign \rangle \texttt{;} \mid \langle FuncCall \rangle \texttt{;} \mid \langle IOStmt \rangle \texttt{;} \mid \texttt{pass;} \\
\langle IfStmt \rangle & ::= & \texttt{if} \ \langle BExp \rangle \ \texttt{then} \ \langle StmtSeq \rangle \ \texttt{else} \ \langle StmtSeq \rangle \ \texttt{fi} \\
 & & \mid \texttt{if} \ \langle BExp \rangle \ \texttt{then} \ \langle StmtSeq \rangle \ \texttt{fi} \\
\langle WhileStmt \rangle & ::= & \texttt{while} \ \langle BExp \rangle \ \texttt{do} \ \langle StmtSeq \rangle \ \texttt{od} \\
\langle AssertStmt \rangle & ::= & \{\langle Spec \rangle\}\texttt{;} \\
\langle Decl \rangle & ::= & \langle Ident \rangle \ \texttt{:} \ \langle Type \rangle \\
\langle Assign \rangle & ::= & \langle Ident \rangle \ \texttt{:=} \ \langle AExp \rangle \\
\langle ListAssign \rangle & ::= & \langle IndexedVal \rangle \ \texttt{:=} \ \langle AExp \rangle \\
\langle IOStmt \rangle & ::= & \texttt{write} \ \langle Ident \rangle \mid \texttt{writeline} \ \langle Ident \rangle
\end{array}
$$

Functions

$$
\begin{array}{rcl}
\langle FuncDecl \rangle & ::= & \texttt{function} \ \langle Ident \rangle \texttt{(} \langle ParamListOpt \rangle \texttt{)} \ \texttt{->} \ \langle FuncType \rangle \ \texttt{::} \ \langle FuncStmtSeq \rangle \ \texttt{end} \\
\langle ParamListOpt \rangle & ::= & \epsilon \mid \langle ParamList \rangle \\
\langle ParamList \rangle & ::= & \langle Param \rangle \ (\texttt{,} \ \langle Param \rangle)^* \\
\langle Param \rangle & ::= & \langle Ident \rangle \ \texttt{:} \ \langle FuncType \rangle \\
\langle FuncStmtSeq \rangle & ::= & \langle FuncStmt \rangle^*{}^{[3]} \\
\langle FuncStmt \rangle & ::= & \langle FuncTermStmt \rangle \mid \langle FuncIfStmt \rangle \mid \langle FuncWhileStmt \rangle \mid \langle AssertStmt \rangle \mid \texttt{;} \\
\langle FuncTermStmt \rangle & ::= & \langle Decl \rangle \texttt{;} \mid \langle Assign \rangle \texttt{;} \mid \langle ListAssign \rangle \texttt{;} \mid \langle FuncCall \rangle \texttt{;} \mid \texttt{pass;} \mid \langle ReturnStmt \rangle \texttt{;} \\
\langle FuncIfStmt \rangle & ::= & \texttt{if} \ \langle BExp \rangle \ \texttt{then} \ \langle FuncStmtSeq \rangle \ \texttt{else} \ \langle FuncStmtSeq \rangle \ \texttt{fi} \\
 & & \mid \texttt{if} \ \langle BExp \rangle \ \texttt{then} \ \langle FuncStmtSeq \rangle \ \texttt{fi} \\
\langle FuncType \rangle & ::= & \langle Type \rangle \mid \texttt{List} \ [\langle Type \rangle, \langle IntLit \rangle] \\
\langle FuncWhileStmt \rangle & ::= & \texttt{while} \ \langle BExp \rangle \ \texttt{do} \ \langle FuncStmtSeq \rangle \ \texttt{od} \\
\langle ReturnStmt \rangle & ::= & \texttt{return} \ \langle AExp \rangle
\end{array}
$$

Return Rule Although not explicitly stated in its grammar, every possible execution `SelVeri` function body must end with a `return` statement. Mostly, our operational semantics will ensure that. However, if the user does not put a `return` statement at the end of the function body; instead of throwing a compilation error, the compiler automatically puts one, returning the default value of the functions return-type. Additionally, as our syntax suggests, a `return` statement can occur only in a function body. These facts will play an important role in our correctness of compilation proofs, therefore we simply name them as the *return rule*.

Function Calls

$$\begin{array}{rcl} \langle \mathit{FuncCall} \rangle & ::= & \texttt{call}\langle \mathit{Ident} \rangle \texttt{(}\langle \mathit{ArgListOpt} \rangle \texttt{)} \\ \langle \mathit{ArgListOpt} \rangle & ::= & \epsilon \mid \langle \mathit{ArgList} \rangle \\ \langle \mathit{ArgList} \rangle & ::= & \langle \mathit{AExp} \rangle \ (\texttt{,}\ \langle \mathit{AExp} \rangle)^* \end{array}$$

Programs

$$\langle \mathit{Program} \rangle \quad ::= \quad (\langle \mathit{FuncDecl} \rangle)^* \ \langle \mathit{StmtSeq} \rangle$$

Semicolon Discipline Declarations, assignments, list assignments, function-call statements, `pass`, and `return` must end with a semicolon. Statements of the form `if` $\cdots$ `fi` and `while` $\cdots$ `od` must not be followed by a semicolon.

2 Semantics

2.1 Important notions for formalizing `SelVeri`

2.1.1 Types

Let **Type** be the set of all possible types. For the current implementation of `SelVeri`, an element of **Type** can be either **Int**, **Float** or a **List** of any dimension, including one of the former two, which can be interpreted as a dependent type. More formally:

- **Int** : **Type**
- **Float** : **Type**
- **List** : $(\Pi t : \mathbf{BasicType} \,.\, (\Pi d : \mathbb{N}^+ \,.\, (\Pi \vec{n} : \mathbb{N}^d \,.\, \mathbf{Type})))$

where $\mathbf{BasicType} := \{\mathbf{Int}, \mathbf{Float}\}$. Similarly, we define $\mathbf{ListType} := \mathbf{Type} \setminus \mathbf{BasicType}$

For convenience, we define the projection function $\pi_d : \mathbb{N}^d \to \mathbb{N}^{d-1}$:

$$\pi_d : (n_1, \cdots, n_d) \mapsto (n_2, \cdots, n_d)$$

and throughout this text, we refer the ith component of $\vec{n}$ as n_i.

2.1.2 Immediates

Let **Imm** be the set of syntactic immediate values supported by `SelVeri`, e.g. 5 or 3.14, and **Var** be the infinite set of all syntactic variables For each type, we need an evaluation function that translates syntactic immediates into the corresponding semantic values.

$$\begin{aligned} \mathcal{I} &: \mathbf{Imm} \cap \mathbf{Int} \to \mathbb{Z} \\ \mathcal{F} &: \mathbf{Imm} \cap \mathbf{Float} \to \mathbb{R} \\ \mathcal{L}_{\mathbf{List}[t,\, d,\, \vec{n}]} &: \mathbf{Imm} \cap \mathbf{List}[t, d, \vec{n}] \to A^{n_1} \end{aligned}$$

where A is the codomain of $\mathcal{L}_{\mathbf{List}[t,\, d\,-\,1,\, \pi_d(\vec{n})]}$. The reader should notice that:

- $\mathbf{List}[t, 0, ()] = t$
- $\mathcal{L}_{\mathbf{Int}} = \mathcal{I}$ and $\mathcal{L}_{\mathbf{Float}} = \mathcal{F}$

For simplicity, we also define $\mathcal{V} := \bigcup_{t \in \mathbf{Type}} \mathcal{L}_t$

2.1.3 Scope

Let **Scope** be the set of all scopes and **AExp** be the set of all arithmetic expressions. Each scope is a representation of 'known' variables in a given block of statements including the variables declared until program execution reaches the corresponding block. Formally,

$$\forall s \in \mathbf{Scope} \,.\, s : \mathbf{AExp} \to \mathbf{Type}$$

Given any scope, s, immediates are mapped to the corresponding types; e.g. $s(5) = \mathbf{Int}$,
$s([1,2,3]) = \mathbf{List}[\mathbf{Int}, 1, 3]$, as expected. Additionally, given any non-atomic arithmetic expression a, we have

$$s(a) = \begin{cases} \mathbf{Int} & \text{if } s(a') = \mathbf{Int} \text{ for all subterms } a' \text{ of } a \\ \mathbf{Float} & \text{if } s(a') = \mathbf{Float} \text{ for some subterm } a' \text{ of } a \end{cases}$$

Note that a composite arithmetic expression cannot have a non-basic type in *SelVeri* as we do not allow a list literal to interact with arithmetic operators.

Initially, each scope starts its lifetime as an empty scope, denoted as $\tilde{s}$,

$$\forall x \in \mathbf{Var} \,.\, \tilde{s}(x) = \mathbf{Void}$$

where $\mathbf{Void} : \mathbf{Type}$ is the empty type.

To simplify the formalization of list-types, given any scope s, we enforce the following implication:

$$s(x) = \mathbf{List}[t, d, \vec{n}] \implies \forall i \in \{0, \ldots, n-1\} \,.\, s(x\,\texttt{[}\,i\,\texttt{]}) = \mathbf{List}[t, d-1, \pi_d(\vec{n})]$$

We call this implication as *recursive declaration principle*. As `SelVeri` syntax does not allow identifiers including `[` or `]` characters, it is not possible to create any scope contradicting the recursive declaration.

Moreover, each scope comes along with a (possibly empty) `parent_scope`. If s is a scope, we denote its immediate child as s_c. In the `SelVeri` implementation, a variable is searched in the `parent_scope`, if it cannot be found in the current scope. When a variable is reached by a parent scope, then a change to its type (which is only possible for special variable `retvar`, see section **2.1.5**) is made directly to the parent scope. However, for simplicity, we omit this special case in our configurations.

Finally, each scope comes with a unique id, which is utilized by `SelVerifier` to differentiate variables with same name that are declared in different scopes.

2.1.4 Program states

Let **State** be the set of all possible program states and **Val** [4] be the set of all semantic values. Each state is in fact a map from **Var** and **Scope** to **Val**, representing the value of a given variable in the specified scope. Formally,

$$\forall \sigma \in \textbf{State} \,.\, \sigma : \textbf{Var} \times \textbf{Scope} \to \textbf{Val}$$

Each program starts with an empty state, $\tilde{\sigma}$ in which each declared variable has the default value of this type, e.g. 0 for **Int**, 0.0 for **Float** and [0, 0, 0] for **List[Int, 1, 3]**. If the specific type is unknown in a given context, the default value of the corresponding type is denoted as **0**.

Also, similar to scopes, program states also have parents. A child program state of σ is denoted as σ_c. When a variable is reached by a parent scope, then its value must be reached by the parent program state. The change must be directly made in the parent program state, in case of an assignment. However, for simplicity, we omit this special case in our configurations.

Additionally, each program state carries an `error_flag` that is set if and only if an error occurs during the execution of the program. A cause for this can be using a variable before declaring it. In order to embed this misusage as an error into our semantics, we add the following rule to our program states:

$$\forall \sigma \in \textbf{State} \,.\, \sigma(x, s) = \bot \quad \text{if } s(x) = \textbf{Void}.$$

Before moving to the next section, reader should be aware of an abuse of notation we use for convenient representation of erroneous states:

- σ represents a program state without any errors.
- $\hat{\sigma}$ represents a program state with an error.
- $\underline{\sigma}$ represents any program state (may be error-free or erroneous).

[4] The reader can notice that $ran\mathcal{I}$, $ran\mathcal{F}$ and $ran\mathcal{L}$ are all subsets of **Val**.

2.1.5 Functions

Let **FuncName** be the infinite set of all valid `SelVeri` function names. To store the information of all function definitions in a given program, we introduce the concept of the *function environment*, denoted as Γ:

$$\Gamma : \textbf{FuncName} \rightarrow (\textbf{Var} \times \textbf{Type})^n \times \textbf{Type} \times \textbf{Code}$$

where n is the arity of the input function and **Code** is the set of all syntactically-correct `SelVeri` programs. During the compilation stage, compiler populates Γ with a single-pass through the function definitions.

For example, if one defines the function `f` as in the following way:

```
function f(x:Int, y:Float, z:List[Float,1,2]) -> Int :: S_f end
```

where $S_{\mathtt{f}}$ is some function statement sequence, then

$$\Gamma(\mathtt{f}) = ((\mathtt{x},\ \mathtt{Int}), (\mathtt{y},\ \mathtt{Float}), (\mathtt{z},\ \mathtt{List[Float,1,2]})),\ \mathtt{Int},\ S_{\mathtt{f}})$$

On the other hand, if `g` is an undefined function, then $\Gamma(\mathtt{g}) = \emptyset$.

Moreover, we introduce a special, dynamically-typed variable `retvar` which holds the return-value of the last called function. Initially, its type is **Void** and its value is $\perp$. `retvar` is a reserved keyword in `SelVeri`, meaning that no user-defined identifier can have this name. An example usage would be:

```
w:Int; call f(5, 3.14, [42, 42]); w := retvar;
```

Assuming that, with the inputs above, execution of the body of `f` does not result in an error state, our operational semantics will ensure that variable `w` holds the result of the computation of `f` (the value returned from `f` to be exact).

2.1.6 List sizes

`SelVeri` supports multi-dimensional lists and to be able to formalize them correctly we define the following helper function size : **Type** $\rightarrow \mathbb{N}$:

$$size(t) = \begin{cases} 0 & \text{if } t \in \textbf{BasicType} \\ \Pi_{i=1}^{d} n_i & \text{if } t = \textbf{List}[t', d, \vec{n}] \text{ for some } t' \in \textbf{BasicType} \end{cases}$$

In a type declaration like $\textbf{List}[t, d, \vec{n}]$, definition of **List** type enforces $t \in$ **BasicType**, so above definition covers all possible t values.

2.1.7 I/O Operations

`SelVeri` programs can interact with the input and output streams. This is necessary due to the philosophy of `SelVeri`, since verification requires reasoning over programs that depend on external inputs rather than fixed constants. These interactions are done utilizing tools available to the abstract machine[5].

[5] Refer to SelVerIR document for details of the abstract machine.

Write Operations. Write operations behave as expected. The string representation of an arithmetic expression is printed after evaluating it under the current state and scope. Formally, for an expression a, the abstract machine outputs $\mathcal{A}(a, \sigma, s)$.

Read as an Arithmetic Expression. Unlike the `write` operation, `read` is not treated as a statement, and it is evaluated as an arithmetic expression. This allows it to be composed naturally inside larger expressions, such as:

$$x := \texttt{read()} + (\texttt{read()} * 2)$$

To support this, we define all possible sequences of input streams as **InputStream**:

$$\iota \in \textbf{InputStream} \implies \exists n \in \mathbb{N}.\ \iota = [v_0, v_1, v_2, \ldots, v_n] \in \textbf{Imm}^{n}$$ [6]

which represents the sequence of values provided by the environment.

[6] Here, n represents the number of inputs provided by the user through the execution of the program. Normally it is equal to the number of occurences of the `read()` statement.

2.2 Semantics of arithmetic expressions

We define the arithmetic valuation function $\mathcal{A}$, representing the semantics of the arithmetic expressions in `SelVeri`.

$$\mathcal{A} : \mathbf{AExp} \times \mathbf{State} \times \mathbf{Scope} \times \mathbf{InputStream} \to \mathbf{Val}$$

$$
\begin{aligned}
\mathcal{A}(x, \sigma, s, \iota) &= \mathcal{I}(x) \quad \text{if } x \in \mathbf{Imm} \wedge s(x) = \mathbf{Int} \\
\mathcal{A}(x, \sigma, s, \iota) &= \mathcal{F}(x) \quad \text{if } x \in \mathbf{Imm} \wedge s(x) = \mathbf{Float} \\
\mathcal{A}(x, \sigma, s, \iota) &= \text{size}(\mathbf{List}[t, d, \vec{n}]) : \mathcal{L}_{\mathbf{List}[t,d,\vec{n}]}(x) \quad \text{if } x \in \mathbf{Imm} \wedge s(x) = \mathbf{List}[t, d, \vec{n}] \\
\mathcal{A}(x, \sigma, s, \iota) &= \sigma(x, s) \quad \text{if } x \in \mathbf{Var} \\
\mathcal{A}(lst[i_1] \dots [i_m], \sigma, s, \iota) &= \mathcal{A}(lst[\mathcal{A}(i_1, \sigma, s, \iota)] \dots [\mathcal{A}(i_m, \sigma, s, \iota)], \sigma, s, \iota) \quad \text{if } lst \in \mathbf{Var} \\
\mathcal{A}(\texttt{len}(x), \sigma, s, \iota) &= \begin{cases} 0 & \text{if } x \in \mathbf{Var} \wedge s(x) \in \mathbf{BasicType} \\ \mathcal{A}(\_x\_\texttt{len}\_1, \sigma, s, \iota) & \text{if } x \in \mathbf{Var} \wedge s(x) \notin \{\mathbf{Void}, \mathbf{Int}, \mathbf{Float}\} \\ \bot & \text{otherwise} \end{cases} \\
\mathcal{A}(a_1 + a_2, \sigma, s, \iota) &= \begin{cases} \mathcal{A}(a_1, \sigma, s, \iota) + \mathcal{A}(a_2, \sigma, s, \iota) & \text{if } \mathcal{A}(a_1, \sigma, s, \iota) \neq \bot \wedge \mathcal{A}(a_2, \sigma, s, \iota) \neq \bot \\ \bot & \text{if } \mathcal{A}(a_1, \sigma, s, \iota) = \bot \vee \mathcal{A}(a_2, \sigma, s, \iota) = \bot \end{cases} \\
\mathcal{A}(a_1 * a_2, \sigma, s, \iota) &= \begin{cases} \mathcal{A}(a_1, \sigma, s, \iota) \cdot \mathcal{A}(a_2, \sigma, s, \iota) & \text{if } \mathcal{A}(a_1, \sigma, s, \iota) \neq \bot \wedge \mathcal{A}(a_2, \sigma, s, \iota) \neq \bot \\ \bot & \text{if } \mathcal{A}(a_1, \sigma, s, \iota) = \bot \vee \mathcal{A}(a_2, \sigma, s, \iota) = \bot \end{cases} \\
\mathcal{A}(a_1 - a_2, \sigma, s, \iota) &= \begin{cases} \mathcal{A}(a_1, \sigma, s, \iota) - \mathcal{A}(a_2, \sigma, s, \iota) & \text{if } \mathcal{A}(a_1, \sigma, s, \iota) \neq \bot \wedge \mathcal{A}(a_2, \sigma, s, \iota) \neq \bot \\ \bot & \text{if } \mathcal{A}(a_1, \sigma, s, \iota) = \bot \vee \mathcal{A}(a_2, \sigma, s, \iota) = \bot \end{cases} \\
\mathcal{A}(a_1/a_2, \sigma, s, \iota) &= \begin{cases} \mathcal{A}(a_1, \sigma, s, \iota)/\mathcal{A}(a_2, \sigma, s, \iota) & \text{if } \mathcal{A}(a_2, \sigma, s, \iota) \neq 0 \wedge \mathcal{A}(a_1, \sigma, s, \iota) \neq \bot \wedge \mathcal{A}(a_2, \sigma, s, \iota) \neq \bot \\ \bot & \text{if } \mathcal{A}(a_2, \sigma, s, \iota) = 0 \vee \mathcal{A}(a_1, \sigma, s, \iota) = \bot \vee \mathcal{A}(a_2, \sigma, s, \iota) = \bot \end{cases} \\
\mathcal{A}(\text{read}(), \sigma, s, i : \iota) &= \mathcal{V}(i) \quad \text{if } i \in \mathbf{Imm} \wedge s(x) \in \mathbf{BasicType}
\end{aligned}
$$

Table 1: The semantics of arithmetic expressions

Although not explicitly stated in the table above, given any binary operation, operators are required to be of the same type (except for the cases of implicit coercion [7] handled by the compiler) and can only be of *basic types* [8]. Otherwise, the result of the evaluation is $\bot$. The same rules apply to the next section (boolean expressions) as well.

[6] If (a_1/a_2) has no subterm of type **Float**, then the / symbol represents the integer division; otherwise, it represents the division of the field $\mathbb{R}$.

[7] An example is multiplying an integer with a float. In such cases, compiler converts the integer to a float with the same value. Conversion of integers to floats is the only possibility for an implicit coercion, as for the current implementation of `SelVeri`.

[8] The only basic types are **Int** and **Float** for the current implementation of `SelVeri`.

2.3 Semantics of boolean expressions

Let **BExp** be the set of all boolean expressions. We define the arithmetic valuation function $\mathcal{B}$, representing the semantics of the boolean expressions in `SelVeri`.

$$\mathcal{B} : \mathbf{BExp} \times \mathbf{State} \times \mathbf{Scope} \to \{0, 1, \bot\}$$

$$
\begin{aligned}
\mathcal{B}(\texttt{true}, \sigma, s) &= 1 \\
\mathcal{B}(\texttt{false}, \sigma, s) &= 0 \\
\mathcal{B}(a, \sigma, s)\ ^{9} &= \begin{cases} 1 & \text{if } \mathcal{A}(a, \sigma, s) \neq \mathbf{0} \\ 0 & \text{if } \mathcal{A}(a, \sigma, s) = \mathbf{0} \\ \bot & \text{if } \mathcal{A}(a, \sigma, s) = \bot \end{cases} \\
\mathcal{B}(a_1 \texttt{ = } a_2, \sigma, s) &= \begin{cases} 1 & \text{if } \mathcal{A}(a_1, \sigma, s) = \mathcal{A}(a_2, \sigma, s) \\ 0 & \text{if } \mathcal{A}(a_1, \sigma, s) \neq \mathcal{A}(a_2, \sigma, s) \\ \bot & \text{if } \mathcal{A}(a_1, \sigma, s) = \bot \vee \mathcal{A}(a_2, \sigma, s) = \bot \end{cases} \\
\mathcal{B}(a_1 \texttt{ <= } a_2, \sigma, s) &= \begin{cases} 1 & \text{if } \mathcal{A}(a_1, \sigma, s) \leq \mathcal{A}(a_2, \sigma, s) \\ 0 & \text{if } \mathcal{A}(a_1, \sigma, s) > \mathcal{A}(a_2, \sigma, s) \\ \bot & \text{if } \mathcal{A}(a_1, \sigma, s) = \bot \vee \mathcal{A}(a_2, \sigma, s) = \bot \end{cases} \\
\mathcal{B}(\texttt{not } b, \sigma, s) &= \begin{cases} 1 & \text{if } \mathcal{B}(b, \sigma, s) = 0 \\ 0 & \text{if } \mathcal{B}(b, \sigma, s) = 1 \\ \bot & \text{if } \mathcal{B}(b, \sigma, s) = \bot \end{cases} \\
\mathcal{B}(b_1 \texttt{ and } b_2, \sigma, s) &= \begin{cases} 1 & \text{if } \mathcal{B}(b_1, \sigma, s) = 1 \text{ and } \mathcal{B}(b_2, \sigma, s) = 1 \\ 0 & \text{if } \mathcal{B}(b_1, \sigma, s) = 0 \text{ or } \mathcal{B}(b_2, \sigma, s) = 0 \\ \bot & \text{if } \mathcal{B}(b_1, \sigma, s) = \bot \text{ or } \mathcal{B}(b_2, \sigma, s) = \bot \end{cases}
\end{aligned}
$$

Table 2: Semantics of boolean expressions

[9] To have this valuation rule, we need $\mathbf{AExp} \subseteq \mathbf{BExp}$ which is the case for `SelVeri`. What this rule means is, every arithmetic expression can also be interpreted as a boolean expression

2.4 Structural operational semantics of the statements

$[\text{decl}_{\text{os}}]$	$\langle x : t, \sigma, s\rangle \to \langle \sigma[x \mapsto \mathbf{0}],\ s[x \mapsto t]\rangle \quad \text{if } t \in \mathbf{BasicType} \wedge s(x) = \mathbf{Void}$
$[\text{decl}^{\text{list}}_{\text{os}}]$	$\langle lst : \mathbf{List}[t, d, \vec{a}], \sigma, s\rangle \to \langle \sigma[lst \mapsto \mathbf{0}\,,\ \_lst\_\texttt{len\_i} \mapsto \mathcal{A}(a_i, \sigma, s)],\ s[lst \mapsto \mathbf{List}[t, d, \mathcal{A}(a_i, \vec{\sigma}, s)]\,,\ \_lst\_\texttt{len\_i} \mapsto \mathbf{Int}]\rangle$ for $i \in \{1, \ldots, d\}$ and if $t \in \mathbf{BasicType} \vee s(x) = \mathbf{Void}$
$[\text{decl}^{\perp}_{\text{os}}]$	$\langle x : t, \sigma, s\rangle \to \hat{\sigma} \quad \text{if } t \notin \mathbf{Type} \vee s(x) \neq \mathbf{Void}$
$[\text{ass}_{\text{os}}]$	$\langle x \texttt{ := } a, \sigma, s\rangle \to \sigma[x \mapsto \mathcal{A}(a, \sigma, s)] \quad \text{if } \mathcal{A}(a, \sigma, s) \neq \perp \wedge \mathcal{A}(a, \sigma, s) \in ran\mathcal{L}_{s(x)}$
$[\text{ass}^{\text{list}}_{\text{os}}]$	$\langle lst\texttt{[}i_1\texttt{]} \ldots \texttt{[}i_m\texttt{]} \texttt{ := } a,\ \sigma,\ s\rangle \to \sigma[lst\texttt{[}\mathcal{A}(i_1, \sigma, s)\texttt{]} \ldots \texttt{[}\mathcal{A}(i_m, \sigma, s)\texttt{]} \mapsto \mathcal{A}(a, \sigma, s)]$ if $\mathcal{A}(a, \sigma, s) \neq \perp \wedge \mathcal{A}(a, \sigma, s) \in ran\mathcal{L}_{s(lst\texttt{[}i_1\texttt{]}\ldots\texttt{[}i_m\texttt{]})}$
$[\text{ass}^{\perp}_{\text{os}}]$	$\langle x \texttt{ := } a, \sigma, s\rangle \to \hat{\sigma} \quad \text{if } \mathcal{A}(a, \sigma, s) = \perp \vee \mathcal{A}(a, \sigma, s) \notin ran\mathcal{L}_{s(x)}$
$[\text{pass}_{\text{os}}]$	$\langle \texttt{pass}, \sigma, s\rangle \to \sigma$
$[\text{comp}^{1}_{\text{os}}]$	$\frac{\langle S_1, \sigma, s\rangle \to \langle S_1', \underline{\sigma}', s'\rangle}{\langle S_1; S_2, \sigma, s\rangle \to \langle S_1'; S_2, \underline{\sigma}', s'\rangle}$ $\qquad$ $[\text{comp}^{2}_{\text{os}}]$ $\quad \frac{\langle S_1, \sigma, s\rangle \to \underline{\sigma}'}{\langle S_1; S_2, \sigma, s\rangle \to \langle S_2, \underline{\sigma}', s\rangle}$
$[\text{scope}_{\text{os}}]$	$\langle \texttt{endscope}, \sigma_c, s_c\rangle \to \langle \sigma, s\rangle \quad \text{where } \sigma, s \text{ are parents of } \sigma_c, s_c$
$[\text{if}^{1}_{\text{os}}]$	$\langle \texttt{if } b \texttt{ then } S_1 \texttt{ else } S_2 \texttt{ fi}, \sigma, s\rangle \to \langle S_1\texttt{;endscope}, \tilde{\sigma_c}, \tilde{s_c}\rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = 1$
$[\text{if}^{0}_{\text{os}}]$	$\langle \texttt{if } b \texttt{ then } S_1 \texttt{ else } S_2 \texttt{ fi}, \sigma, s\rangle \to \langle S_2\texttt{;endscope}, \tilde{\sigma_c}, \tilde{s_c}\rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = 0$
$[\text{if}^{\perp}_{\text{os}}]$	$\langle \texttt{if } b \texttt{ then } S_1 \texttt{ else } S_2 \texttt{ fi}, \sigma, s\rangle \to \hat{\sigma} \quad \text{if } \mathcal{B}(b, \sigma, s) = \perp$
$[\text{while}^{1}_{\text{os}}]$	$\langle \texttt{while } b \texttt{ do } S \texttt{ od}, \sigma, s\rangle \to \langle S\texttt{;endscope;while } b \texttt{ do } S \texttt{ od}, \tilde{\sigma_c}, \tilde{s_c}\rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = 1$
$[\text{while}^{0}_{\text{os}}]$	$\langle \texttt{while } b \texttt{ do } S \texttt{ od}, \sigma, s\rangle \to \sigma \quad \text{if } \mathcal{B}(b, \sigma, s) = 0$
$[\text{while}^{\perp}_{\text{os}}]$	$\langle \texttt{while } b \texttt{ do } S \texttt{ od}, \sigma, s\rangle \to \hat{\sigma} \quad \text{if } \mathcal{B}(b, \sigma, s) = \perp$
$[\text{call}_{\text{os}}]$	$\langle \texttt{call } f(a_1, \ldots, a_n), \sigma, s\rangle \to \langle x_1 : t_1\texttt{;}x_1 \texttt{:=} v_1\texttt{ ; } \ldots \texttt{ ;}x_n : t_n\texttt{;}x_n \texttt{:=} v_n\texttt{ ;}S_f,\ \tilde{\sigma}', \tilde{s}'\rangle$ if $t_i = s(x_i)$ for $i = 1, 2, \ldots, n$ where $\Gamma(f) = (((x_1', t_1), \ldots, (x_n', t_n)), t', S_f)$ and $v_j \in \mathbf{Imm} \wedge \mathcal{A}(v_j, \tilde{\sigma}', \tilde{s}') = \mathcal{A}(x_j, \sigma, s)$ for $j = 1, 2, \ldots, n$
$[\text{ret}_{\text{os}}]$	$\langle \texttt{return } a, \sigma', s'\rangle \to \langle \sigma[\texttt{retvar} \mapsto \mathcal{A}(a, \sigma', s')],\ s[\texttt{retvar} \mapsto s'(a)]\rangle$ if the return-type of the function called is equal to $s'(a)$ where σ and s are of returning function's caller.
$[\text{write}_{\text{os}}]$	$\langle \texttt{write}(x), \sigma, s\rangle \to \langle \sigma,\ s\rangle$ where the abstract machine emits the value $\mathcal{A}(x, \sigma, s)$
$[\text{writeln}_{\text{os}}]$	$\langle \texttt{writeline}(x), \sigma, s\rangle \to \langle \sigma,\ s\rangle$ where the abstract machine emits the value $\mathcal{A}(x, \sigma, s)$ and a newline
$[\text{err}_{\text{os}}]$	$\langle S, \hat{\sigma}, s\rangle \to \hat{\sigma}$

Table 3: Small-step operational semantics for statements

depending on the context and its truth value is determined by an equality to zero check, similar to the `C` programming language.

Note that not all possible configurations are included in the table above. In any case which is absent from the table, program state is transitioned into $\hat{\sigma}$.

Moreover, for each if-rule, an `if`-statement without the `else`-part, namely a statement of the form "`if` b `then` S `fi`" can be treated as "`if` b `then` S `else pass fi`"

3 Determinism of `SelVeri`

In order to rigorously establish that `SelVeri` is a deterministic language, we must prove that its small-step operational semantics transition relation ($\rightarrow$) is a partial function.

Theorem (Determinism): For any statement S, state σ, and scope s, if $\langle S, \sigma, s\rangle \rightarrow \gamma_1$ and $\langle S, \sigma, s\rangle \rightarrow \gamma_2$, then $\gamma_1 = \gamma_2$.

Proof: The proof proceeds by structural induction on the statement S. Before evaluating the statements, we first establish the determinism of expressions.

Step 1: Determinism of Expressions
According to our semantics, the arithmetic valuation function $\mathcal{A}$ is defined as a deterministic mathematical function mapping a tuple of (**AExp**, **State**, **Scope**, **InputStream**) to exactly one value in **Val** (Table 1). Similarly, the boolean valuation function $\mathcal{B}$ maps to exactly one value in $\{0, 1, \perp\}$ (Table 2). Because these evaluations are strictly defined as functions, they inherently yield exactly one result for any given configuration.

Step 2: Mutual Exclusivity of Statement Rules
We proceed by structural induction on S, showing that for any given configuration, at most one operational semantic rule can apply (Table 3).

- **Control Flow (`if` and `while`):** The transition rules for `if` ($[\text{if}^1_{\text{os}}]$, $[\text{if}^0_{\text{os}}]$, $[\text{if}^{\perp}_{\text{os}}]$) and `while` ($[\text{while}^1_{\text{os}}]$, $[\text{while}^0_{\text{os}}]$, $[\text{while}^{\perp}_{\text{os}}]$) depend strictly on the evaluation of $\mathcal{B}(b, \sigma, s)$. Since $\mathcal{B}$ deterministically evaluates to exactly 1, 0, or $\perp$, these premises are mutually exclusive.
- **Assignments:** The basic assignment rules ($[\text{ass}_{\text{os}}]$ and $[\text{ass}^{\perp}_{\text{os}}]$) possess mutually exclusive premises. $[\text{ass}_{\text{os}}]$ requires $\mathcal{A}(a, \sigma, s) \neq \perp \wedge \mathcal{A}(a, \sigma, s) \in ran\mathcal{L}_{s(x)}$. The rule $[\text{ass}^{\perp}_{\text{os}}]$ fires if and only if this exact condition is false ($\mathcal{A}(a, \sigma, s) = \perp \vee \mathcal{A}(a, \sigma, s) \notin ran\mathcal{L}_{s(x)}$). The same mutual exclusivity applies to list assignments.
- **Declarations:** The declaration rules ($[\text{decl}_{\text{os}}]$ and $[\text{decl}^{\perp}_{\text{os}}]$) rely on mutually exclusive type and scope checks. $[\text{decl}_{\text{os}}]$ applies if $t \in$ **BasicType** $\wedge$ $s(x) =$ **Void**, whereas $[\text{decl}^{\perp}_{\text{os}}]$ acts as the deterministic fallback if $t \notin$ **Type** $\vee$ $s(x) \neq$ **Void**.
- **Error States:** If a state is already erroneous ($\hat{\sigma}$), the catch-all error rule $[\text{err}_{\text{os}}]$ deterministically transitions the program into $\hat{\sigma}$. Furthermore, as stated previously, any configuration absent from the operational semantics table also deterministically transitions into $\hat{\sigma}$.

Because the expressions are deterministic and the transition rules for statements are mutually exclusive, it is impossible for more than one operational semantic rule to apply to any given configuration. Therefore, $\gamma_1 = \gamma_2$, and the semantics of `SelVeri` are strictly deterministic. ■

4 Turing-completeness of `SelVeri`

To formally establish the Turing-completeness of `SelVeri`, we demonstrate that the language can compute all μ-recursive (general recursive) functions. By the Church-Turing thesis, any computational model capable of representing the basic functions and closed under composition, primitive recursion, and minimization is Turing-complete.

Theorem: Every general recursive function can be computed by a valid `SelVeri` program.

Proof: We show that `SelVeri` constructs, underpinned by the operational semantics defined in **section 2**, are sufficient to implement the foundational elements of μ-recursive functions. We utilize the `Int` basic type, which semantically maps to the unbounded infinite set of integers ($\mathbb{Z}$) via the valuation function $\mathcal{I}$.

1. The Basic Functions

- **Zero Function** $Z(x) = 0$: Supported natively by returning the integer immediate 0.

```
function Z(x : t) -> Int ::
    return 0;
end
```

- **Successor Function** $S(x) = x+1$: Supported by the arithmetic valuation function $\mathcal{A}(x \texttt{ + } \texttt{1}, \sigma, s, \iota)$.

```
function S(x: Int) -> Int ::
    return x + 1;
end
```

- **Projection Function** $P_i^k(x_1, \ldots, x_k) = x_i$: Supported natively by the parameter passing and scope rules ($s(x_i) =$ **Int**) of `SelVeri` function declarations. For example, $P_i^k(x_1, \ldots, x_k) = x_i$ is implemented as:

```
function P_i_k(x1 : t_i, ..., x_k : t_k) -> t_i ::
    return x_i;
end
```

  Similarly, list index accessing (`lst[i]`) can be interpreted as another example of projection function in `SelVeri`.

2. Closure Under Composition
Given a function $h(\vec{x}) = f(g_1(\vec{x}), \ldots, g_m(\vec{x}))$, `SelVeri` natively handles composition through sequential function calls and the dynamically-typed `retvar` mechanism. Each inner function g_i can be evaluated using a `call` $g_i(\ldots)$ statement. By the [call_{os}] and [ret_{os}] rules, the result is safely stored in the `retvar` scope, assigned to an intermediate variable, and subsequently passed as an argument to the outer function f. For example, a composition $h(x_1, \ldots x_n) = f(g_1(x_1^1 \ldots x_{m_1}^1), \ldots g_k(x_1^k \ldots x_{m_k}^k))$ is implemented as:

```
function h(x_1: t_1, ..., x_n : t_n) -> t_r ::
    res_g1 : t_r_1;
    .
    .
    .
    res_gk : t_r_k
    final_res : t_r;

    call g1(x_1_1, ..., x_1_m_1);
    res_g1 := retvar;
    .
    .
    .
    call gk(x_k_1, ..., x_k_m_k);
    res_gk := retvar;

    call f(res_g1, ..., res_gk);
    final_res := retvar;

    return final_res;
end
```

3. Closure Under Primitive Recursion
A function defined by primitive recursion takes the form:

$$h(\vec{x}, 0) = f(\vec{x})$$
$$h(\vec{x}, y+1) = g(\vec{x}, y, h(\vec{x}, y))$$

This is realized iteratively via `SelVeri`'s `while` loop. Because the `Int` type models unbounded integers natively without overflow, we can simulate the recursive steps from 0 up to y:

```
function h(x_1 : t_1, ..., x_k : t_k y: Int) -> t_r ::
    counter : Int;
    accum : t_r;

    call f(x_1 ... x_k);
    accum := retvar;
    counter := 0;

    while counter < y do
        call g(x_1 ... x_k, counter, accum);
        accum := retvar;
        counter := counter + 1;
    od

    return accum;
end
```

The loop's structural soundness is strictly guaranteed by the mutually exclusive $[\text{while}^1_{\text{os}}]$ and $[\text{while}^0_{\text{os}}]$ semantic rules evaluating $\mathcal{B}(\texttt{counter} < y, \sigma, s)$.

Another way of showing primitive recursiveness would be directly constructing a recursive function:

```
function h(x_1 : t_1, ..., x_k : t_k y : Int) -> t_r ::
    if y = 0 then
        call f(x_1 ... x_k);
        return retvar;

    h_rec_result : t_r;
    call h(x_1 ... x_k, y-1);
    h_rec_result = retvar;

    call g(x_1 ... x_k, y-1, h_rec_result);
    return retvar;
end
```

The base case of `y = 0` and strict decrease of `y` in non-base cases ensure the completion of the function call.

4. Closure Under Minimization (μ-operator)
The minimization operator $\mu y[f(\vec{x}, y) = 0]$ finds the smallest y such that $f(\vec{x}, y) = 0$. This unbounded search introduces the potential for partial functions (non-halting programs), a strict requirement for full Turing-completeness. In `SelVeri`, this translates to an unbounded `while` loop checking an arithmetic evaluation interpreted as a boolean:

```
function mu_f(x_1 : t_1 ... x_k : t_k) -> t_y ::
    y : Int;
    res: Int;

    y := 0;
    call f(x_1 ... x_k, y);
    res := retvar;

    while not (res = 0) do
        y := y + 1;
        call f(x_1 ... x_k, y);
        res := retvar;
    od

    return y;
end
```

By the operational semantics of boolean expressions established in Table 2, the `while` condition evaluates the equality constraint $\mathcal{B}(\texttt{not}\ (\texttt{res} = 0), \sigma, s)$, causing the loop to terminate and return y if and only if such a y exists.

Conclusion: Because `SelVeri` represents the basic functions and is closed under composition, primitive recursion, and unbounded minimization using its explicitly defined structural operational semantics, it is capable of computing any μ-recursive function. Thus, `SelVeri` is Turing-complete. ■